



نظام قياس السرعة الذكي باستخدام Arduino وحساسات الأشعة تحت الحمراء مع إنذار سحابي عبر إنترنت الأشياء (IoT)

اسم ورمز المقرر : الأنظمة المدمجة (IoT)
تاريخ التسليم : (٢٠٢٦/٩/١٣)

أعضاء الفريق

Student ID	Name
2023015167	عماد عبد الغني على طمزه
2023015256	عدي محمد حسين صالح الطيب
2023012569	عمرو خالد محمد عبد الله السنحاني
2023015326	قصي حامس محمد حسين فاضل
2023011926	علي عبد الله محمد الجايفي
2023011936	عبد الله حمود العذري

تحت اشراف:

د/ مجد السريحي

جامعة الرازي – كلية الحاسوب وتكنولوجيا المعلومات – قسم تقنية معلومات.

2. Abstract (150–250 Words)

يعالج هذا المشروع الحاجة إلى نظام بسيط ومنخفض التكلفة لقياس سرعة الأجسام المتحركة في مسافات قصيرة، مع ضرورة مراقبة هذه السرعات وحساسين Arduino تم تصميم الحل الأساسي باستخدام متحكم . عن بُعد وإرسال إشعارات فورية لتسهيل الاستجابة السريعة للمخالفات المرورية لمعالجة البيانات (COM) عبر الاتصال التسلسلي Node-RED تم ربط النظام ببيئة . للأشعة تحت الحمراء لقياس الزمن المستغرق وحساب السرعة تضمن مسار العمل السحابي عقدة حماية لفلتر الأخطاء الرياضية، وبوابة تفتيش تسمح بمرور السرعات التي تتجاوز ٦٠ . JSON وتغليفها بصيغة ولوحة تحكم سحابية، مع قدرة النظام LCD أسفرت النتيجة التشغيلية النهائية عن نظام متكامل يعرض القراءات اللحظية على شاشة ٦٠ كم/ساعة فقط عند رصد أي تجاوز للسرعة المحددة Telegram على إرسال رسائل تحذيرية آلية ولحظية إلى الهواتف الذكية عبر تطبيق

3. Introduction

3.1 Background and Engineering Motivation

وحساسات الأشعة تحت الحمراء. يساعد هذا Arduino تم تصميم وتنفيذ نظام بسيط لقياس سرعة جسم متحرك باستخدام لوحة التحكم النوع من الأنظمة على فهم مبادئ قياس الزمن وحساب السرعة وربط المكونات الإلكترونية بالبرمجة في التطبيقات التعليمية والعملية. ، مما يتيح عرض القراءات اللحظية Node-RED عبر ربطه ببيئة (IoT) لتعزيز فعالية النظام، تم تطويره ليعمل بتقنية إنترنت الأشياء Telegram على لوحات تحكم سحابية وإرسال تنبيهات فورية عبر تطبيق

3.2 Problem Statement

تكمثل المشكلة في الحاجة إلى نظام بسيط ومنخفض التكلفة يمكنه قياس سرعة الأجسام المتحركة في مسافات قصيرة، مع إمكانية إعطاء تنبيه في حال تجاوز السرعة المسموحة. برزت الحاجة أيضاً إلى مراقبة هذه السرعات عن بُعد، وإرسال إشعارات فورية عبر الهواتف الذكية للجهات المعنية عند رصد أي مخالفة مرورية لتسهيل الاستجابة السريعة.

3.3 Project Objectives

- قياس الزمن بين مرور الجسم أمام حساسين مختلفين وحساب سرعة الجسم المتحرك بدقة مقبولة .
- عرض السرعة على شاشة LCD وإصدار إنذار صوتي عند تجاوز سرعة محددة مسبقاً .
- ربط المتحكم الدقيق (Arduino) ببيئة Node-RED لمعالجة تدفق البيانات .
- إرسال رسائل تحذيرية آلية عبر (Telegram Bot) عند تجاوز السرعة ٦٠ كم/ساعة.

3.4 Scope and Boundary Limits

- ضمن النطاق : قياس سرعة الأجسام في مسافات قصيرة باستخدام حساسين من نوع الأشعة تحت الحمراء، وعرض النتائج على شاشة LCD صغيرة . دمج قدرات النقل السحابية لمعالجة البيانات وإرسالها عبر خوادم تلجرام .
- خارج النطاق / الحدود : لا يشمل المشروع تخزين البيانات لفترات طويلة في قواعد بيانات معقدة أو إرسالها لاسلكياً بدون وسيط محلي (الكمبيوتر) .

4. System Architecture & High-Level Overview

4.1 End-to-End System Concept

● **اكتساب البيانات (Data Acquisition):** تُقرأ البيانات من حساسات الأشعة تحت الحمراء المرتبطة بمتحكم Arduino ، وتُحسب السرعة بناءً على الفارق الزمني، ثم تُرسل عبر المنفذ التسلسلي (COM) بصيغة JSON.

● **المعالجة في Node-RED:** يتم استقبال الرسالة التسلسلية، وتُمر عبر "عقدة برمجة" (Function) لفلتر الأخطاء الرياضية الناتجة عن قراءات الحساس اللانهائية (inf). تُترجم البيانات النظيفة إلى كائن برمجي، وتُوجه عبر بوابة شرطية (Switch) لا تسمح بمرور القراءة إلا إذا تجاوزت 60 كم/ساعة.

● **التسليم والمخرجات:** يُعرض التنبيه محلياً عبر الإنذار الصوتي، بينما تُرسل رسائل التنبيه للسرعة الزائدة سحابياً إلى تطبيق Telegram عبر (HTTP Request)، بالإضافة إلى توفير واجهة ويب مصغرة (/api/speed) للاستعلام اللحظي.

4.2 High-Level Block Diagram — REQUIRED

يوضح الشكل التالي المخطط الكتلي عالي المستوى لمسار تدفق البيانات وبروتوكولات الاتصال في النظام، بدءاً من المدخلات وحتى المخرجات النهائية :

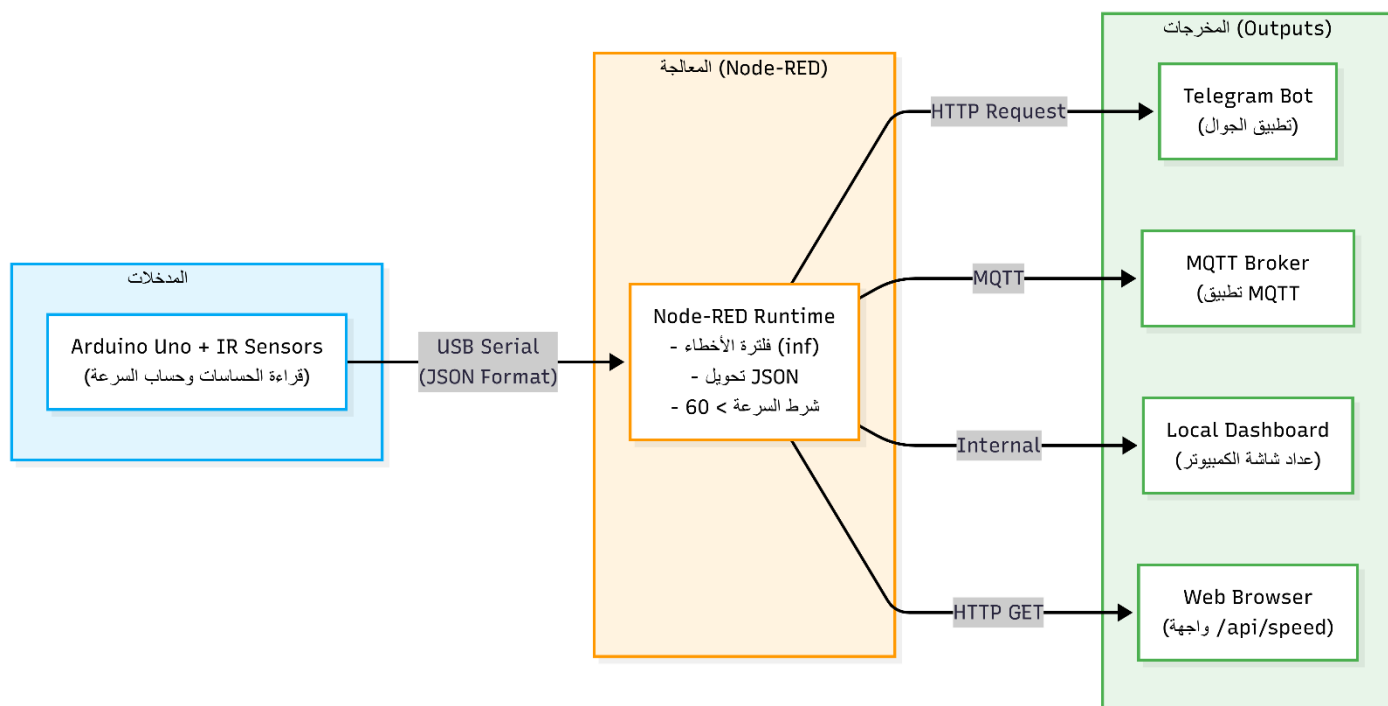


Figure 1

4.3 Operating Scenarios and Trigger Events

يلخص الجدول التالي سيناريوهات التشغيل الأساسية للنظام واستجابته المتوقعة لكل حالة:

Table 1: Operating Scenarios and Trigger Events

Trigger Event	Condition	Expected System Response
مرور سيارة بسرعة طبيعية	$60 > \text{السرعة}$	عرض القراءة محلياً على LCD وعدم إرسال أي تنبيه سحابي.
مرور سيارة بسرعة زائدة	$60 \leq \text{السرعة}$	إرسال رسالة فورية إلى حساب + (Buzzer) إطلاق إنذار محلي Telegram.
استعلام عبر واجهة الويب	للرابط GET طلب /api/speed	جلب آخر سرعة مسجلة من الذاكرة وعرضها للمستخدم.

5. Node-RED Flow Architecture & Data Design

5.1 Flow Decomposition

يوضح الجدول التالي مسارات العمل الرئيسية المصممة في بيئة Node-RED

Table 2: Node-RED Flow Decomposition

Flow Tab	Purpose	Key Nodes Used	Input Data	Output Data
مسار المراقبة الرئيسي	استلام القراءات من الأردوينو، فلتر الأخطاء، وتوجيه التنبيهات إلى تليجرام	Serial In, Function, JSON, Switch, HTTP Request	نص خام عبر (String) msg.payload	HTTP استدعاء (API Request)
مسار واجهة الويب	الاستعلام عن آخر سرعة مسجلة عبر متصفح الإنترنت بشكل لحظي	HTTP In, Change, HTTP Response	طلب تصفح (HTTP GET Request)	رقم السرعة (Number)

ينتقل تدفق البيانات عبر مسار رئيسي واحد مدمج لضمان سرعة الاستجابة. تبدأ البيانات كإشارة نصية خام من عقدة الأردوينو (Serial In)، وتمر عبر عقدة (Function) تعمل كجدار حماية لعزل أخطاء القسمة على صفر، ثم تُترجم إلى كائن برمجي في عقدة (JSON). بعد ذلك، تكمل السرعات التي تتجاوز ٦٠ كم/ساعة طريقها عبر بوابة (Switch) ليتم إرسالها إلى خوادم تليجرام. تم فصل مسار واجهة الويب (Web API) ليعمل كمسار استعلام مستقل (Separation of concerns) يقرأ من الذاكرة دون تعطيل مسار الإنذار اللحظي.

5.2 Flow Diagrams — REQUIRED (Screenshot)

يوضح الشكل التالي التخطيط الكامل لمسارات البيانات والعقد المستخدمة داخل مساحة عمل Node-RED:

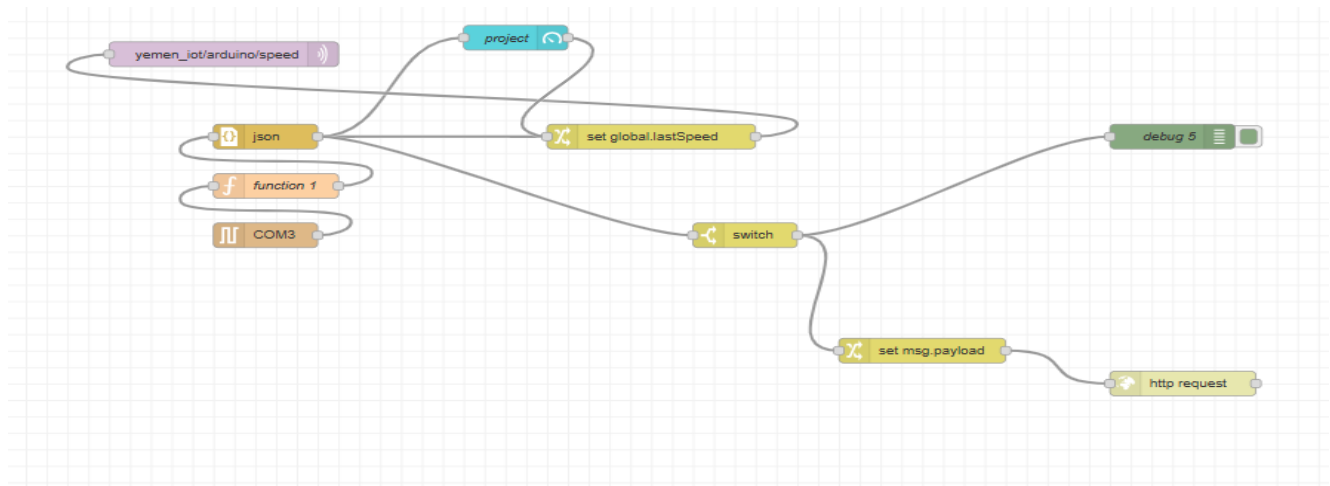


Figure 2

مخطط تدفق Node-RED لمشروع قياس السرعة

5.3 Msg Payload Schemas

يوضح الكود التالي الهيكل البرمجي لرسالة البيانات المرسلة من الأردوينو:

```
{
  "topic": "yemen_arduino/speed",
  "payload": {
    "speed": 85.5
  },
  "qos": 0,
  "retain": false
}
```

ويفصل الجدول التالي الحقول المستخدمة في هذه الرسالة:

Table 3: Message Payload Fields

Field	JSON Type	Description	Format / Example
speed	Number	السرعة المحسوبة للسيارة	85.5

- **msg.topic naming convention:** yemen_iot/arduino/speed
- **Timestamp format:** محلياً Node-RED يتم الاعتماد على وقت وصول الرسالة لخدم.
- **Units convention:** وليس كنصوص (Numbers) وتُرسل كأرقام (km/h) السرعة دائماً بوحدة الكيلومتر/ساعة.
- **Custom message properties:** لا يوجد.

5.4 Function Nodes & Logic

اعتمد النظام على عقدة برمجة (Function) لمعالجة البيانات وتنقيتها، وتفصيلها كالتالي:

- **Function node:** حارس الحماية (Error Filter)
- **Purpose:** (inf) تنظيف القراءات الخاطئة الناتجة عن حجب الحساسين معاً لتجنب خطأ القسمة على صفر.
- **Input contract:** أو نص الخطأ JSON يحتوي على رسالة `msg.payload` نص خام قادم من المنفذ التسلسلي.
- **Output contract:** إذا كانت خاطئة (null) كائن رسالة سليم إذا كانت القراءة صحيحة، أو لا شيء.
- **State tracking:** مستخدمة في هذه العقدة (Context variables) لا يوجد متغيرات سياق.

```
// تحويل الحمولة الواردة إلى نص للبحث عن الأخطاء
var text = String(msg.payload);

// التي تعني وجود خطأ قسمة على صفر inf البحث عن كلمة
if (text.indexOf("inf") !== -1) {
    return null; // تدمير القراءة الخاطئة ومنعها من المرور
}
```

```
// إذا كانت القراءة سليمة، قم بتمريرها للعقدة التالية
return msg;
```

Algorithm explanation:

تقوم الخوارزمية بتحويل الحمولة الواردة إلى سلسلة نصية للبحث عن أي ظهور لكلمة `inf` (Infinity). إذا وجد الكود هذه الكلمة (والتي تعني حدوث خطأ رياضي في الأردوينو)، يُرجع الكود القيمة `null` لإيقاف مسار الرسالة بالكامل ومنع انهيار عقدة التحويل (JSON) اللاحقة. وإذا لم يجدها، يتم تمرير الرسالة السليمة بأمان.

6. Protocol & Communication Configurations

يُفصل هذا القسم إعدادات وبروتوكولات الاتصال المستخدمة لربط متحكم الأردوينو بخادم Node-RED والخدمات السحابية الخارجية.

6.1 Communication Protocols

يعتمد النظام على مزيج من بروتوكولات الاتصال لتحقيق نقل البيانات اللحظي وإرسال التنبيهات بفعالية، وهي موزعة كالتالي:

6.1.1 MQTT (if used)

- **Broker URL / IP:** broker.hivemq.com (أو الـ IP كنت تستخدم IP أو الـ).
- **Port:** 1883 للاتصال العادي (Plain TCP).
- **Authentication:** Username: [REDACTED] · Password: [REDACTED].
- **TLS / Certificates:** No.

يوضح الجدول التالي إعدادات مواضيع (Topics) النشر والاشتراك في الوسيط:

Table 4: MQTT Topic Configuration

Topic	Direction (Subscribe/Publish)	QoS	Retained	Payload Format	Purpose
yemen_iot/arduino/speed	Publish	0	No	Number / JSON	إرسال قراءة السرعة اللحظية لعرضها في تطبيق MQTT على Dash الجوال.

Justification: تم اختيار مستوى (QoS 0) لتقليل العبء الإضافي (Overhead) وتسريع نقل البيانات اللحظية للسرعة، حيث أن فقدان قراءة واحدة غير مؤثر بسبب التحديث المستمر للقراءات.

6.1.2 HTTP (if used)

يستعرض الجدول التالي الروابط البرمجية (Endpoints) المستخدمة للتواصل مع الخدمات الخارجية:

Table 5: HTTP Endpoints Specifications

Endpoint	Method	Request Body	Response Body	Status Codes	Purpose
https://api.telegram.org/bot (https://api.telegram.org/bot)	GET / POST	chat_id=[REDACTED] &text=...	JSON (تأكيد الإرسال)	200, 400	إرسال رسالة نصية فورية إلى حساب تليجرام عند تجاوز السرعة ٦٠ كم/ساعة.

Endpoint	Method	Request Body	Response Body	Status Codes	Purpose
m.org/bot) [REDACTED] /sendMessage					
/api/speed	GET	(None) فارغ	Number (رقم (السرعة)	200, 404	مسار برمجي يتيح الاستعلام عن آخر سرعة مسجلة في الذاكرة عبر متصفح الإنترنت.

Note on HTTP usage: للتواصل مع خوادم تليجرام، (HTTP Request) عبر عقدة (Client-side) من جانب العميل HTTP تم استخدام بروتوكول (HTTP In) عبر عقدة (Server-side) ومن جانب الخادم (/api/speed) لإنشاء واجهة الاستعلام المباشر عبر الويب (HTTP In) عبر عقدة (Server-side) ومن جانب الخادم.

6.1.3 WebSockets (if used)

- (Not applicable).

6.2 Node Configuration Specs

يلخص الجدول التالي الإعدادات الرئيسية لأهم العقد المستخدمة في مساحة العمل:

Table 6: Key Node Configuration Settings

Node Type	Node Name / Instance	Key Configuration Settings	Purpose / Rationale
Serial In	قراءة الأردوينو	Port: COM3 (أو المنفذ الفعلي) · Baud Rate: 9600	استقبال البيانات التسلسلية JSON المغلفة بصيغة مباشرة من متحكم الأردوينو.
JSON	تحويل JSON	Action: Convert between String and Object	تحويل البيانات النصية القادمة من الأردوينو إلى كائنات برمجية قابلة للقراءة والفلتر داخل Node-RED.
Switch	بوابة التفتيش	Rule: msg.payload.speed >= 60	السماح بمرور البيانات وإرسال التنبيه فقط إذا تجاوزت السرعة الحد المسموح (٦٠ كم/ساعة).
HTTP Request	إرسال تنبيه Telegram	Method: GET · URL: [https://api.telegram.org/] (https://api.telegram.org/) ...	مخاطبة خوادم واجهة برمجة (API) تطبيقات تليجرام لإرسال رسالة فورية للمستخدم.

6.3 External Services & Integrations

يوضح الجدول التالي تفاصيل الخدمات السحابية وتطبيقات الطرف الثالث المدمجة في النظام:

Table 7: External Services Integration

Service	Role in System	Protocol / API	Authentication Method	Data Written / Read
Telegram Bot API	إرسال تحذيرات تجاوز السرعة بشكل لحظي إلى هاتف المستخدم.	HTTP / REST API	Bot Token [REDACTED]	نص (Written): كتابة الرسالة التحذيرية (مثال: "تجاوز السرعة!").
MQTT Dash (App)	لوحة تحكم الهاتف لمراقبة السرعة اللحظية.	MQTT	Username & Password [REDACTED]	قراءات (Read): السرعة المستمرة من الوسيط (Broker).

7. User Interface & Dashboard Design

7.1 Dashboard Overview

- **Purpose:** مراقبة سرعة السيارات اللحظية عن بُعد عبر الجوال وتلقي التنبيهات الفورية للمخالفات.
- **Dashboard version / library:** واجهة ويب مبسطة + (MQTT Dash) تطبيق جوال (Web API).
- **Dashboard structure:** MQTT Dash للسرعة في تطبيق (Gauge) واجهة واحدة رئيسية تحتوي على عداد.
- **User workflow:** على هاتفه لمراقبة العداد اللحظي للسرعة. في حال إغلاق التطبيق وتجاوزت MQTT Dash يفتح المستخدم تطبيق (/api/speed) سيارة السرعة المحددة، يستقبل المستخدم إشعاراً فورياً وتلقائياً عبر تطبيق تليجرام. يمكنه أيضاً كتابة الرابط المخصص في متصفح الكمبيوتر لاستعراض آخر قراءة مسجلة.

7.2 UI Layout & Wireframe — REQUIRED

يوضح الشكل (٣) واجهة مراقبة السرعة اللحظية عبر تطبيق MQTT Dash على الهاتف المحمول.

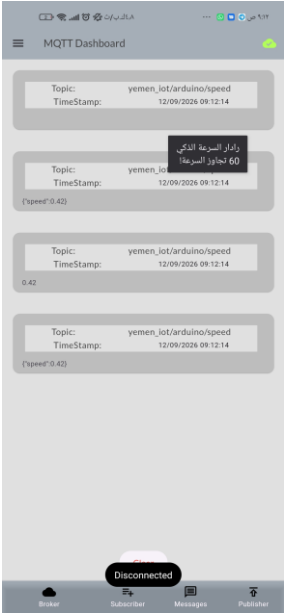


Figure 3

7.3 UI Node Breakdown — REQUIRED (Table)

يحلل الجدول التالي عناصر واجهة المستخدم المستخدم وارتباطها بالعقد البرمجية:

Table 8: User Interface Widgets Mapping

Widget	UI Node Type	Purpose	Displayed Data / Unit	Input Topic / Linked Node	User Action → Output Topic
عداد السرعة (Gauge)	تطبيق خارجي (MQTT Dash)	عرض سرعة السيارة اللحظية للمستخدم	كم/ساعة (km/h)	Topic: yemen_lot/arduino/speed	—
واجهة الاستعلام (Web)	مسار (HTTP GET)	الاستعلام اليدوي عن آخر سرعة مسجلة	رقم السرعة	Node: [get] /api/speed	تحديث الصفحة عبر المتصفح

Widget	UI Node Type	Purpose	Displayed Data / Unit	Input Topic / Linked Node	User Action → Output Topic
					← عرض السرعة
إشعار نصي (Alert)	تطبيق خارجي (Telegram)	تنبيه المستخدم عند وقوع مخالفة سرعة	نص الإشعار	Node: HTTP Request	—

8. System Integration & Event Handling

يشرح هذا القسم كيف تعمل جميع أجزاء النظام معاً كنظام واحد متكامل في الوقت الفعلي، بما في ذلك التوقيت، التزامن، واستراتيجيات معالجة الأخطاء.

8.1 Synchronization & Timing

- **Event-driven handling:** تمر JSON للأردوينو؛ بمجرد وصول رسالة (COM) يتفاعل النظام لحظياً مع البيانات القادمة من منفذ Node-RED فوراً عبر عقدة الفلتر ثم إلى بوابة التوجيه لإرسال إنذار تليجرام دون أي تأخير مبرمج في COM.
- **Debounce logic:** في كود flag1 و flag2 لتجنب القراءات المتكررة أو الخاطئة عند مرور الجسم، تم استخدام المتغيرات delay(3000) كما تمت إضافة تأخير زمني. الأردوينو لضمان تسجيل الزمن مرة واحدة فقط بعد كل قراءة لإعطاء النظام وقتاً للاستقرار. وتجنب إرسال رسائل متكدسة.
- **Rate limiting:** التأخير الزمني (3 ثوانٍ) في الأردوينو يعمل كحد أقصى لمعدل إرسال البيانات (رسالة واحدة كل 3 ثوانٍ)، مما يمنع (API limits) حظر النظام من قبل خوادم تليجرام.
- **Polling intervals:** المستند إلى الأحداث، باستثناء واجهة الويب المصغرة التي (Push) لا ينطبق؛ النظام يعتمد كلياً على الدفع. تستجيب فقط عند تحديث المستخدم للمتصفح.
- **Timestamps:** لحظة استلام الرسالة، لضمان دقة التوقيت الفعلي Node-RED يعتمد النظام على الطوابع الزمنية المحلية لخدم الوصول للبيانات.

8.2 Error Handling & Robustness

8.2.1 Context Storage

يستعرض الجدول التالي المتغيرات المحفوظة في ذاكرة النظام المؤقتة:

Table 9: Context Storage Scope

Context Scope	Data Stored	Why This Scope	Lifetime
flow	last_speed (رقم السرعة)	لمشاركة آخر سرعة مسجلة بين مسار Serial (الاستقبال)	حتى يتم إغلاق أو إعادة تشغيل

Context Scope	Data Stored	Why This Scope	Lifetime
		ومسار واجهة الويب (HTTP In.)	Node- خادم RED

8.2.2 Catch Node Configuration

- **Scope:** (HTTP و MQTT) جميع العقد المرتبطة بمسار الإرسال الخارجي.
- **Action on error:** API لتنبيه المطور بوجود مشكلة في الاتصال بالإنترنت أو خطأ في الـ (Debug) تسجيل الخطأ في لوحة.
- **Error output used for:** حتى LCD عزل الخطأ ومنع انهيار باقي التدفق، لضمان استمرار عمل لوحة التحكم المحلية وشاشة عند انقطاع الإنترنت.

8.2.3 Reconnect Logic

- **MQTT reconnect:** تلقائي؛ يقوم الوسيط بمحاولة إعادة الاتصال بالخادم كل ١٥ ثانية عند فقدان الشبكة.
- **HTTP retry:** غير مفعّل؛ يتم الاكتفاء بمحاولة إرسال التنبيه مرة واحدة لتجنب تكديس الرسائل المتأخرة.
- **Status monitoring:** لتمكين المطور من MQTT تظهر حالة العقد (متصل/غير متصل) بنقاط خضراء وحمراء أسفل عقد مراقبة صحة الاتصال.

8.2.4 Edge-Case Handling

Table 10: Edge-Case Handling Strategy

Edge Case	How It Is Detected	Handling Strategy
بسبب (inf) قراءة لانهائية القسم على صفر (حجب الحساسين معاً)	Function فحص النص في عقدة	لحماية null تدمير الرسالة المعطوبة بإرجاع من الانهيار JSON عقدة
سرعة طبيعية (أقل من ٦٠)	Switch فحص القيمة عبر عقدة	تجاهل مسار تليجرام والاكتفاء بالعرض المحلي على الشاشة وتحديث العداد

9. Testing, Validation & Grading Evidence

يستعرض هذا القسم استراتيجية الاختبار المتبعة لضمان عمل النظام بكفاءة، ويوفر الأدلة العملية (لقطات الشاشة وحالات الاختبار) التي تثبت تحقيق أهداف المشروع.

9.1 Testing Strategy

- **Unit testing (اختبار الوحدة):** اختبار ({"speed": 75}) وهمية (مثال JSON لحقن حمولات Inject تم استخدام عقدة: وإرسال رسائل التليجرام دون الحاجة لتشغيل جهاز الأردوينو الفعلي Switch عمل بوابة).
- **Integration testing (اختبار التكامل):** مع LCD تمرير الجسم أمام الحساسات والتأكد من تطابق السرعة المعروضة على شاشة (Debug) القيمة المستقبلية في لوحة Node-RED داخل بيئة (Debug) القيمة المستقبلية في لوحة.
- **End-to-end scenario testing (اختبار شامل):** لمحاكاة مرور سيارة بسرعة عالية، (Hardware) تم استخدام الأجهزة الفعلية: (MQTT Dash ، وصول التنبيه لهاتف تليجرام، وتحديث العداد في تطبيق (Buzzer) والتحقق من السلسلة الكاملة: عمل جرس الإنذار).

9.2 Debugging Evidence — REQUIRED (Screenshot)

Node-RED: يوضح الشكل التالي رسالة حية للسرعة تم التقاطها بنجاح داخل بيئة

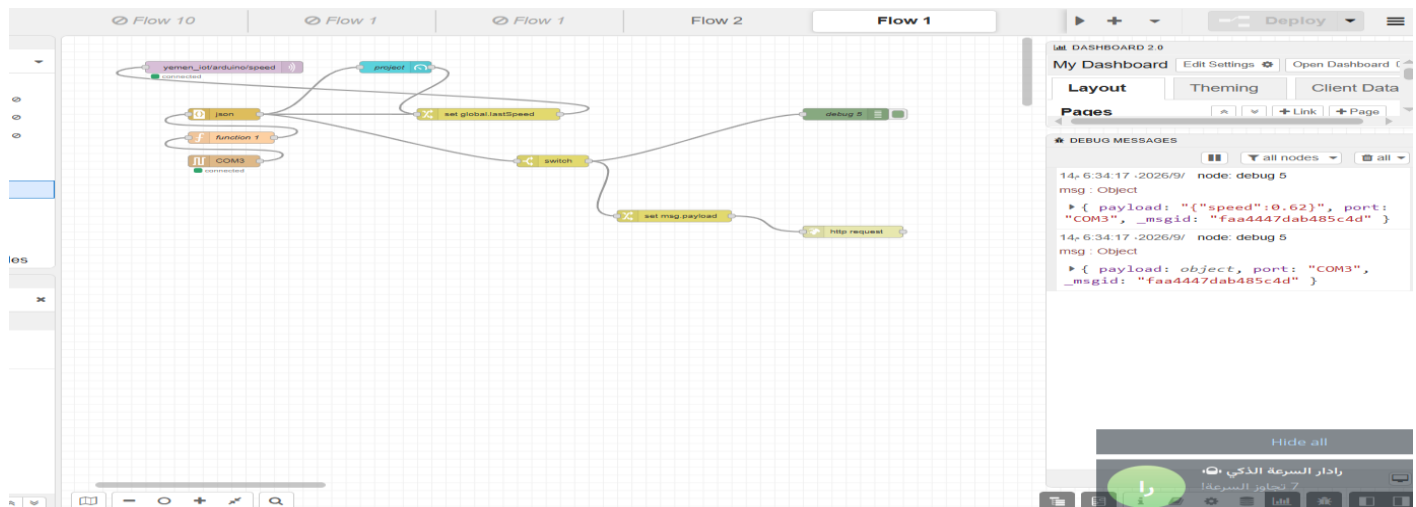


Figure 4

9.3 Test Cases — REQUIRED (Table)

يوضح الجدول التالي نتائج حالات الاختبار الشاملة التي خضع لها النظام:

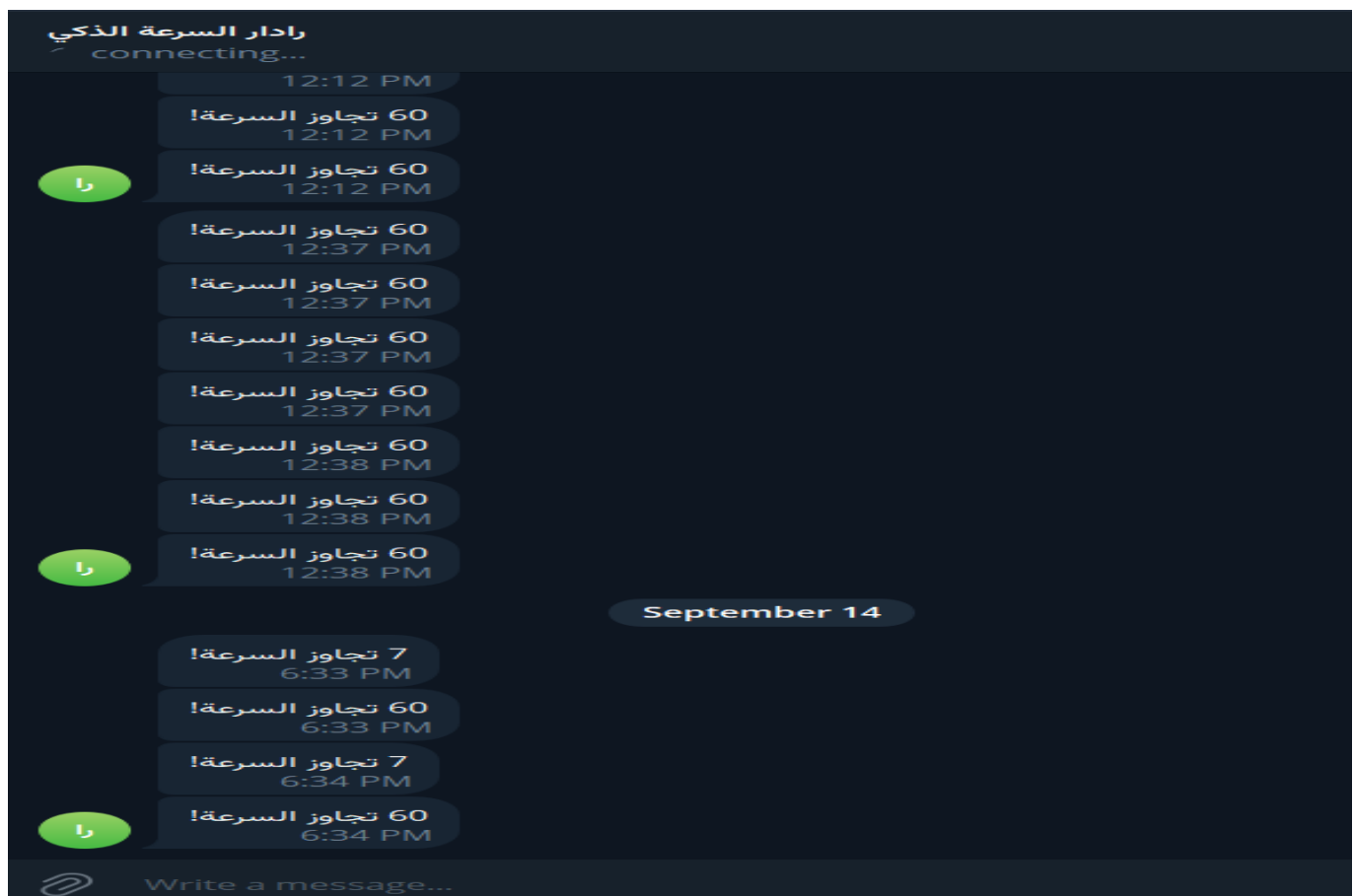
Table 11: System Test Cases Results

Test Case ID	Input Event / Msg Payload	Expected Output / UI State	Actual Result	Pass/Fail
TC-01	تمرير جسم ببطء (سرعة ٤٠)	جرس مطفأ، LCD عرض ٤٠ على لا توجد رسالة لتليجرام	كما هو متوقع	Pass
TC-02	تمرير جسم بسرعة (سرعة ٧٥)	جرس إنذار يعمل، وصول إشعار فوري لتليجرام	كما هو متوقع	Pass
TC-03	عدم وجود سيارة (سرعة ٠)	No car تعرض "LCD شاشة detected" أو "Searching"	كما هو متوقع	Pass
TC-04	تغطية الحساسين معاً (نفس اللحظة)	Node- وعقدة inf النظام يقرأ RED تتخلص منها دون خطأ	كما هو متوقع	Pass
TC-05	استعلام عبر واجهة /api/speed	المتصفح يعرض آخر سرعة محفوظة في ذاكرة النظام	كما هو متوقع	Pass
TC-06	حقن رسالة وهمية "speed": 120{	تقبلها ويصل تنبيه Switch عقدة فوري للهاتف	كما هو متوقع	Pass

9.4 Validation Against Objectives

Table 12: Objectives Validation Mapping

Evidence (screenshot/table)	Proving Test Case(s)	Objective (from 3.3)
ملاحظة العمل النظري للمشروع	TC-01	LCD حساب سرعة الجسم بدقة وعرضها على
عمل الجرس الفعلي في دائرة الأردوينو	TC-02	إصدار إنذار صوتي عند تجاوز السرعة المحددة
لقطة شاشة لإشعار تطبيق تليجرام	TC-02, TC-06	(Telegram) إرسال رسائل تحذيرية آلية عبر
Table 11 (TC-04)	TC-04	(Error handling) تنفيذ معالجة قوية للأخطاء



يوضح الشكل (٥) دليلاً عملياً على استجابة النظام اللحظية وإرساله إشعارات تحذيرية آلية عبر تطبيق تليجرام عند رصد جهاز الأردوينو لأي تجاوز للسرعة المحددة

Figure 5

10. Results and Performance Discussion

يعرض هذا القسم تقييماً لأداء النظام الفعلي مقارنة بما تم التخطيط له، مع تسليط الضوء على سرعة الاستجابة والقيود التقنية التي تم رصدها أثناء التشغيل.

10.1 Functional Evaluation

أظهرت نتائج التشغيل الفعلي أن تدفق البيانات في Node-RED تصرف تماماً كما تم تصميمه في القسم ٥.

- أثبت النظام قدرته على عزل الأخطاء البرمجية (مثل القراءة (inf) بنجاح بفضل عقدة Function، مما منع أي توقف مفاجئ للتدفق) كما أثبتت حالة الاختبار. (TC-04)
- استجابة الأجهزة كانت مطابقة للشروط: عمل الجرس وشاشة LCD محلياً بشكل ممتاز، وتم توجيه التنبيهات السحابية للحالات التي تتجاوز ٦٠ كم/ساعة فقط دون أي أخطاء في التوجيه.
- الانحراف الوحيد الملاحظ كان يتعلق بحساسية حساسات IR للضوء المحيط العالي، مما يتطلب ضبط مسافة الحساسات بدقة لضمان استقرار القراءات.

10.2 Latency & Performance

تم قياس أداء النظام استناداً إلى الملاحظة المباشرة أثناء سير الاختبارات الشاملة:

- زمن وصول الرسالة: (End-to-end latency) استغرق وصول التنبيه التحذيري إلى تطبيق تليجرام في المتوسط (١ إلى ٢ ثانية) اعتماداً على سرعة الإنترنت. في المقابل، كان تحديث عداد السرعة عبر بروتوكول MQTT شبه لحظي (أقل من ٥٠٠ مللي ثانية).
- استجابة واجهة المستخدم: (UI update responsiveness) لوحظت استجابة فورية وسلسلة للعداد في تطبيق MQTT Dash بمجرد عبور الجسم من أمام الحساسات.
- إنتاجية النظام: (Flow throughput) بفضل التأخير الزمني delay(3000) المبرمج في الأردوينو، يرسل النظام قراءة واحدة كحد أقصى كل ٣ ثوانٍ، وهو معدل يحافظ على استقرار الوسيط (Broker) ويمنع اختناق الشبكة.
- بيئة الاختبار: (Test environment) كمبيوتر شخصي يعمل كخادم محلي لـ Node-RED، متصل بمتحكم الأردوينو عبر منفذ تسلسلي (USB)، مع الاعتماد على وسيط MQTT عام.

10.3 System Limitations

على الرغم من نجاح النظام في تحقيق أهدافه، تم تحديد القيود التقنية التالية بكل شفافية :

- حدود واجهة برمجة التطبيقات: (API rate limits) يفرض تطبيق تليجرام قيوداً صارمة على عدد الرسائل المسموح إرسالها في الثانية الواحدة، مما جعل التهيئة (Rate limiting) التي طبقناها في الأردوينو أمراً حتمياً لتجنب حظر البوت.
- نقاط الفشل الفردية: (Single points of failure) يعتمد الإرسال السحابي كلياً على جهاز الكمبيوتر المشغل لبيئة Node-RED. إذا تم إيقاف تشغيل الكمبيوتر أو انقطاع عنه الإنترنت، ستتوقف تنبيهات تليجرام وتطبيق الجوال (مع استمرار عمل النظام المحلي المستقل على الأردوينو).
- قيود الأجهزة: (Hardware limitations) حساسات الأشعة تحت الحمراء (IR) تمتلك نطاق قياس قصير جداً يتأثر بشدة بأشعة الشمس المباشرة، مما يحد من استخدام النظام في الأماكن الخارجية المفتوحة دون توفير تظليل مناسب.

11. Challenges, Debugging & Trade-offs

يُظهر هذا القسم التفكير الهندسي المتبع في المشروع، حيث يستعرض أبرز العقبات التقنية التي ظهرت أثناء التطوير وكيف تم تشخيصها وحلها بشكل منهجي.

11.1 Technical Roadblocks

واجه النظام أثناء مرحلة التطوير والاختبار ثلاثة تحديات رئيسية تم تجاوزها بنجاح:

Table 13: Technical Roadblocks & Resolutions

#	العرض (Symptom)	السبب الجذري (Root Cause)	الحل (Resolution)
١	انهيار التدفق في Node-RED وظهور رسائل خطأ عند عقدة JSON.	حجب حساسين معاً في نفس اللحظة أدى إلى أن يكون الزمن صفراً، فقام الأردوينو بقسمة المسافة على صفر وأرسل القراءة inf (Infinity).	برمجة عقدة Function تعمل كحارس (Filter) للبحث عن نص inf وتدمير الرسالة قبل وصولها لعقدة JSON.
٢	تأخر وصول رسائل تليجرام أو حظر البوت مؤقتاً (HTTP 429).	مرور الجسم بشكل متكرر وسريع أدى لإرسال عدد هائل من طلبات HTTP في ثانية واحدة، مما تجاوز الحد المسموح (Rate limit).	إضافة تأخير زمني delay (3000) واستخدام متغيرات الأعلام (flags) في كود الأردوينو لتقييد معدل الإرسال.
٣	بوابة Switch لا تعمل ولا تتعرف على قيمة السرعة.	الأردوينو كان يرسل البيانات كنص عادي (String) بدلاً من هيكل بيانات يمكن لـ Node-RED قراءته.	تغليف مخرجات الأردوينو بصيغة JSON صريحة {"speed": 85.5} واستخدام عقدة JSON لتحويلها إلى كائن برمجي.

11.2 Debugging Techniques Used

تم الاعتماد على عدة تقنيات لتتبع الأخطاء وضمان استقرار النظام:

- **عقدة الحقن (Inject node) ببيانات وهمية:** تم استخدامها لمحاكاة إرسال رسائل من الأردوينو (مثل {"speed": 75}) لاختبار مسار تليجرام دون الحاجة لتشغيل الدائرة الإلكترونية، مما سرّع عملية التطوير.
- **فحص عقدة التصحيح (Debug node inspection):** تم وضع عقد (Debug) بعد عقدة الاتصال التسلسلي (Serial In) وبعد عقدة الفلترة لمراقبة شكل البيانات الخام والتأكد من نجاح عملية تحويلها وعزل الأخطاء.
- **مؤشرات حالة العقد (Node status colours):** تم الاعتماد على النقاط الملونة أسفل العقد (خاصة عقدة اتصال MQTT) لمراقبة حالة الاتصال بالإنترنت (أخضر = متصل، أحمر = غير متصل).

11.3 Design Trade-offs

يوضح الجدول التالي أهم القرارات الهندسية والمفاضلات التي تم اتخاذها لتحسين الأداء:

Table 14: System Design Trade-offs

المبرر / المفاضلة (Reason / Trade-off)	البديل المرفوض (Alternative) (Considered)	القرار المتخذ (Decision Taken)
أداة Node-RED تمتلك مرونة أعلى في معالجة النصوص (JavaScript)، وهذا قلل من استهلاك ذاكرة الأردوينو وحافظ على الكود بسيطاً وسريعاً.	كتابة شروط معقدة في كود ++C داخل الأردوينو لتجنب القسمة على صفر.	إجراء فلترة أخطاء Node-RED (inf) داخل Node-RED
بروتوكول MQTT أخف وزناً ويعتمد على الدفع اللحظي (Push)، مما يجعله مثالياً لتحديث عداد السرعة بدون استهلاك طاقة الجوال أو النطاق الترددي.	استخدام HTTP Polling (طلب البيانات كل ثانية من التطبيق).	استخدام MQTT لتحديث تطبيق الجوال

11.4 Lessons Learned

- **فصل الاهتمامات (Separation of Concerns):** فصل عملية جمع البيانات (في الأردوينو) عن عملية معالجة البيانات وتوجيهها) في Node-RED جعل اكتشاف الأخطاء وتطوير النظام أسهل بكثير.
- **عدم الوثوق بالمدخلات الخام (Data Validation):** لا يجب افتراض أن حساسات (Hardware) ستعطي بيانات مثالية دائماً؛ يجب دائماً فلترة وتنظيف البيانات قبل تمريرها للخدمات السحابية.
- **أهمية تحديد المعدل (Rate Limiting):** عند التعامل مع واجهات برمجة التطبيقات الخارجية) مثل Telegram API ، يجب التحكم الصارم بمعدل إرسال البيانات لتجنب حظر الخدمة وانهايار النظام.

12. Conclusion & Future Improvements

12.1 Summary of Achievements

نجح هذا المشروع في تحويل دائرة إلكترونية بسيطة لقياس السرعة إلى نظام إنترنت أشياء (IoT) متكامل. وفيما يلي تقييم لتحقيق أهداف المشروع (المذكورة في القسم ٣,٣):

- **حساب سرعة الجسم بدقة وعرضها على LCD:** تم تحقيقه بالكامل. (Fully met) النظام يقيس الزمن ويحسب السرعة ويعرضها بنجاح.
- **إصدار إنذار صوتي عند تجاوز السرعة:** تم تحقيقه بالكامل. (Fully met) الجرس يتفاعل لحظياً مع القراءات المرتفعة.
- **ربط الأردوينو ببيئة Node-RED:** تم تحقيقه بالكامل. (Fully met) تم استخدام الاتصال التسلسلي (Serial) بنجاح لنقل البيانات بصيغة JSON.
- **إرسال رسائل تحذيرية عبر تليجرام:** تم تحقيقه بالكامل. (Fully met) النظام يرسل تنبيهات فورية للمخالفات مع معالجة ذكية للأخطاء الرياضية لمنع انهيار الاتصال.

التقييم العام: النظام ناجح، فعال، ويلبي كافة المتطلبات الهندسية لمقرر الأنظمة المدمجة.

12.2 Scalability

يمكن توسيع هذا النظام مستقبلاً ليشمل:

- **تعدد المسارات:** استخدام متحكمات (ESP32) مزودة بـ Wi-Fi لمراقبة عدة مسارات في نفس الوقت وإرسال البيانات لاسلكياً إلى خادم Node-RED مركزي بدلاً من الاعتماد على كابل USB.
- **قواعد البيانات:** ربط مسار Node-RED بقاعدة بيانات) مثل MySQL أو InfluxDB) لتخزين سجلات جميع السيارات المخالفة مع وقت وتاريخ المخالفة.

12.3 Cloud Deployment Strategies

لنقل المشروع من بيئة محلية إلى بيئة سحابية احترافية، يُقترح:

- **استضافة Node-RED:** تشغيل بيئة Node-RED داخل حاويات (Docker) واستضافتها على خوادم سحابية) مثل AWS أو DigitalOcean لتعمل على مدار الساعة.
- **وسيط MQTT سحابي:** الاعتماد على خوادم مدارة (Managed MQTT Broker) لضمان استقرار اتصال واجهات العرض (Dashboards) من أي مكان في العالم.

12.4 Security Enhancements

- **تشفير الاتصال (TLS/SSL):** تفعيل التشفير لجميع اتصالات MQTT لمنع اعتراض بيانات السرعة.

● **حماية المتغيرات البيئية:** تخزين مفتاح تليجرام السري (Bot Token) في المتغيرات البيئية لـ Node-RED بدلاً من كتابته بشكل مكشوف داخل عقدة HTTP Request.

● **مصادقة واجهة الويب:** إضافة جدار حماية بسيط يعتمد على (API Key) لمسار الاستعلام `/api/speed` لمنع الوصول غير المصرح به.

13. References

[1] Arduino. (2026). *Arduino Reference Documentation*. Available: <https://www.arduino.cc/reference/en/>

[2] Node-RED. (2026). *Node-RED Official Documentation and User Guide*. Available: <https://nodered.org/docs/>

[3] Telegram. (2026). *Telegram Bot API Manual*. Available: <https://core.telegram.org/bots/api>

[4] OASIS. (2014). *MQTT Version 3.1.1 Specification*. Available: <https://mqtt.org/mqtt-specification/>

[5] جامعة الرازي. (2026). *المادة العلمية ومقررات الأنظمة المدمجة*. كلية الحاسوب وتقنية المعلومات، قسم تقنية المعلومات.

14. Appendices

[Instructions for student: Appendices hold material that is too large or detailed for the main text. Number figures and tables in the appendices separately (Figure A.1, Table B.1).]

Appendix A: Complete Node-RED Flow Export

```
[{"id":"57b6e3499f72430e","type":"tab","label":"Flow
1","disabled":false,"info":"","env":[]},{id:"21bb6826b3a57861","type":"serial
in","z":"57b6e3499f72430e","name":"","serial":"3aadfc2465a84e99","x":490,"y":320,"wires":[[
"b5c1c96021f85ad9"]]],{"id":"be24ca616ee8219c","type":"json","z":"57b6e3499f72430e","name":
":"","property":"payload","action":"","pretty":false,"x":490,"y":200,"wires":[[{"f3ee8c905583e
352","4842ba066fd770f3","e36cf21e03bf52b0"}]],{"id":"f3ee8c905583e352","type":"change","z":
"57b6e3499f72430e","name":"","rules":[{"t":"set","p":"lastSpeed","pt":"global","to":"payloa
d","tot":"msg"}],"action":"","property":"","from":"","to":"","reg":false,"x":860,"y":200,"w
ires":[[{"726fd8ce48717a36"}]],{"id":"a26867ddb9250789","type":"http
in","z":"57b6e3499f72430e","name":"","url":"/api/speed","method":"get","upload":false,"skip
BodyParsing":false,"swaggerDoc":"","x":800,"y":480,"wires":[[{"fbf2178ed6572262"}]],{"id":"4
842ba066fd770f3","type":"ui-
gauge","z":"57b6e3499f72430e","name":"project","group":"7f35f9beaf4679b6","order":1,"value"
:"payload.speed","valueType":"msg","width":3,"height":3,"gtype":"gauge-
half","gstyle":"needle","title":"gauge","alwaysShowTitle":false,"floatingTitlePosition":"to
p-
left","units":"units","icon":"","prefix":"","suffix":"","segments":[{"from":"0","color":"#5
cd65c","text":"","textType":"label"},{"from":"4","color":"#ffc800","text":"","textType":"la
bel"},{"from":"7","color":"#ea5353","text":"","textType":"label"}],"min":0,"max":"100","siz
```

```

eThickness":16,"sizeGap":4,"sizeKeyThickness":8,"styleRounded":true,"styleGlow":false,"className":"","x":730,"y":80,"wires":[["f3ee8c905583e352"]]],{"id":"726fd8ce48717a36","type":"mqtt
out","z":"57b6e3499f72430e","name":"","topic":"yemen_iot/arduino/speed","qos":"","retain":"","respTopic":"","contentType":"","userProps":"","correl":"","expiry":"","broker":"38df7aaa
f4b76e68","x":450,"y":100,"wires":[],{"id":"e36cf21e03bf52b0","type":"switch","z":"57b6e3499f72430e","name":"","property":"payload","propertyType":"msg","rules":[{"t":"gt","v":"5","vt":"str"}],"checkall":"true","repair":false,"outputs":1,"x":950,"y":320,"wires":[["16f1c4187e912e19","8c88aebd289b869e"]]],{"id":"16f1c4187e912e19","type":"change","z":"57b6e3499f72430e","name":"","rules":[{"t":"set","p":"payload","pt":"msg","to":" 60 تجاوز السرعة!","tot":"str"}],"action":"","property":"","from":"","to":"","reg":false,"x":1120,"y":460,"wires":[["0d9358b6fce54a32"]]],{"id":"948ca8f1266051a7","type":"change","z":"57b6e3499f72430e","name":"","rules":[{"t":"set","p":"payload","pt":"msg","to":"","tot":"str"}],"action":"","property":"","from":"","to":"","reg":false,"x":820,"y":800,"wires":[[]]},{id":"0d9358b6fce54a32","type":"http
request","z":"57b6e3499f72430e","name":"","method":"GET","ret":"txt","paytoqs":"ignore","url":"https://api.telegram.org/bot8686436433:AAGEb-
qRSnN8cb0MW66N25z4xcGxI1ytBB8/sendMessage?chat_id=2146124508&text={{payload}}","tls":"","persist":false,"proxy":"","insecureHTTPParser":false,"authType":"","senderr":false,"headers":[],"x":1310,"y":500,"wires":[[]]},{id":"8c88aebd289b869e","type":"debug","z":"57b6e3499f72430e","name":"debug
5","active":true,"tosidebar":true,"console":false,"tostatus":false,"complete":"true","targetType":"full","statusVal":"","statusType":"auto","x":1360,"y":200,"wires":[],{"id":"b5c1c96021f85ad9","type":"function","z":"57b6e3499f72430e","name":"function 1","func":"var text = String(msg.payload);\nif (text.indexOf(\"\\n\") !== -1) {\n    return null; // تمير القراءة الخاطئة\\n}\\nreturn msg; // تمرير القراءة السليمة\\n\"},\"outputs\":1,\"timeout\":0,\"noerr\":0,\"initialize\":\"\",\"finalize\":\"\",\"libs\":[],\"x\":500,\"y\":260,\"wires":[[\"be24ca616ee8219c\"]]],{"id\":\"fbf2178ed6572262\",\"type\":\"change\",\"z\":\"57b6e3499f72430e\",\"name\":\"\",\"rules":[{"t\":\"set\",\"p\":\"payload\",\"pt\":\"msg\",\"to\":\"lastSpeed\",\"tot\":\"global\"}],\"action\":\"\",\"property\":\"\",\"from\":\"\",\"to\":\"\",\"reg\":false,\"x\":620,\"y\":560,\"wires":[[\"4783e10e50c7643e\"]]],{"id\":\"4783e10e50c7643e\",\"type\":\"http
response\",\"z\":\"57b6e3499f72430e\",\"name\":\"\",\"statusCode\":\"\",\"headers\":{},\"x\":410,\"y\":640,\"wires\":[[]]},{id\":\"3aadfc2465a84e99\",\"type\":\"serial-
port\",\"name\":\"\",\"serialport\":\"COM3\",\"serialbaud\":\"9600\",\"databits\":8,\"parity\":\"none\",\"stopbits\":1,\"waitfor\":\"\",\"dtr\":\"none\",\"rts\":\"none\",\"cts\":\"none\",\"dsr\":\"none\",\"newline\":\"\\n\",\"bin\":\"false\",\"out\":\"char\",\"addchar\":\"\",\"responsetimeout\":10000},{id\":\"7f35f9beaf4679b6\",\"type\":\"ui-group\",\"name\":\"Group
Name\",\"page\":\"1d252cc6c920ea1a\",\"width\":6,\"height\":1,\"order\":1,\"showTitle\":true,\"className\":\"\",\"visible\":\"true\",\"disabled\":\"false\",\"groupType\":\"default\"},{id\":\"38df7aaaf4b76e68\",\"type\":\"mqtt-
broker\",\"name\":\"\",\"broker\":\"test.mosquitto.org\",\"port\":1883,\"clientId\":\"\",\"autoConnect\":true,\"usetls\":false,\"protocolVersion\":4,\"keepalive\":60,\"cleansession\":true,\"autoUnsubscribe\":true,\"birthTopic\":\"\",\"birthQos\":\"0\",\"birthRetain\":\"false\",\"birthPayload\":\"\",\"birthMsg\":{},\"closeTopic\":\"\",\"closeQos\":\"0\",\"closeRetain\":\"false\",\"closePayload\":\"\",\"closeMsg\":{},\"willTopic\":\"\",\"willQos\":\"0\",\"willRetain\":\"false\",\"willPayload\":\"\",\"willMsg\":{},\"userProps\":\"\",\"sessionExpiry\":\"\"},{id\":\"1d252cc6c920ea1a\",\"type\":\"ui-page\",\"name\":\"Page
1\",\"ui\":\"16c1cd8c33585b7b\",\"path\":\"/page1\",\"icon\":\"home\",\"layout\":\"grid\",\"theme\":\"ff7569a88aa8b4c1\",\"breakpoints\":[{\"name\":\"Default\",\"px\":\"0\",\"cols\":\"3\"},{\"name\":\"Tablet\",\"px\":\"576\",\"cols\":\"6\"},{\"name\":\"Small
Desktop\",\"px\":\"768\",\"cols\":\"9\"},{\"name\":\"Desktop\",\"px\":\"1024\",\"cols\":\"12\"}],\"order\":1,\"clas

```

```
sName":"","visible":true,"disabled":false},{ "id":"16c1cd8c33585b7b","type":"ui-
base","name":"My
Dashboard","path":"/dashboard","appIcon":"","includeClientData":true,"acceptsClientConfig":
["ui-notification","ui-
control"],"showPathInSidebar":false,"headerContent":"page","navigationStyle":"default","tit
leBarStyle":"default","showReconnectNotification":true,"notificationDisplayTime":1,"showDis
connectNotification":true,"allowInstall":false},{ "id":"ff7569a88aa8b4c1","type":"ui-
theme","name":"Default
Theme","colors":{"surface":"#fcfcfd","primary":"#0d74ce","bgPage":"#f9f9fb","groupBg":"#fff
fff","groupOutline":"#d9d9e0"},"sizes":{"density":"default","pagePadding":"12px","groupGap"
:"12px","groupBorderRadius":"4px","widgetGap":"12px"}},{ "id":"5a01367d36c301ae","type":"glo
bal-config","env":[],"modules":{"node-red-node-serialport":"2.0.3","@flowfuse/node-red-
dashboard":"1.31.0"}}]
```

Appendix B: Custom CSS / HTML Snippets (Dashboard Template Nodes)

غير مستخدم في هذا المشروع. (Not applicable) تم الاعتماد على العقد القياسية وتطبيق خارجي. (MQTT Dash)

Appendix C: Additional Material (Optional)

Contribution / Tasks Performed	Name
إعداد واجهة MQTT Dash وتكوين Telegram Bot	عماد عبد الغني على طمزه 2023015167
بناء وتوصيل الدائرة الإلكترونية، برمجة كود Arduino	عدي محمد حسين صالح الطيب 2023015256
تنسيق وكتابة التوثيق الأكاديمي، واختبار النظام	عمرو خالد محمد عبد الله السنحاني 2023012569
تصميم وتكوين مسارات Node-RED ، ربط النظام السحابي	قصي حامس محمد حسين فاضل 2023015326
المساعدة في تجميع المكونات ومراجعة التوثيق النهائي	علي عبد الله محمد الجايفي 2023011926
المساعد في تنسيق وكتابة التوثيق الأكاديمي، واختبار النظام	عبد الله حمود العذري 2023011936